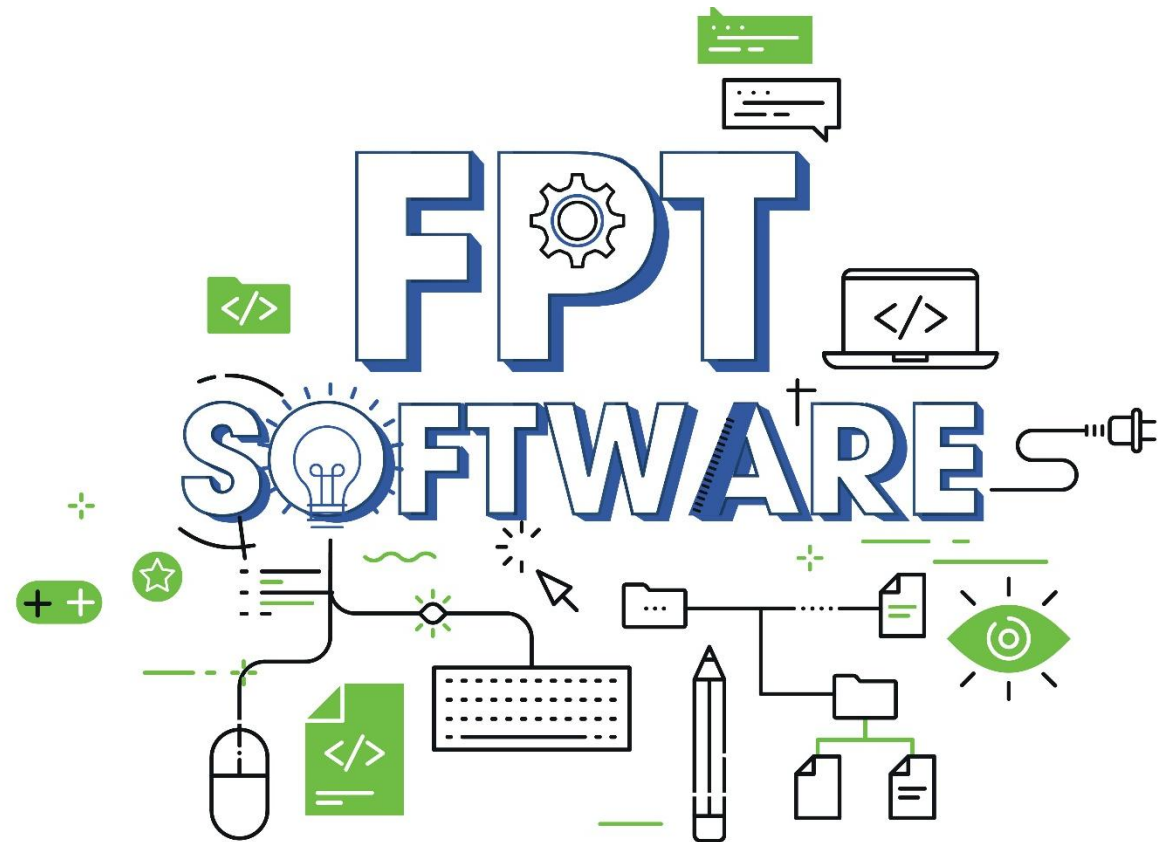


Explore Reactive Programming - 2

Android Advance



Content

- *Advance operator usage*
- *Multi Threading using reactive approach*
- *Kotlin Flow replacement*

Section 1

Advance operator usage

Operator usage in RxJava

In RxJava, an "**operator**" is a crucial component for processing and transforming data within Observable streams. An operator is a function or method used to perform transformations, filtering, combining, or applying other operations to the data within an Observable, resulting in a new Observable.

- Filtering/suppressing operators
- Transforming operators
- Collection operators
- Error handling operators
- Action operators

Filtering/suppressing operators

- ***filter(predicate)***: The *filter()* operator filters items emitted by an Observable based on a provided predicate function. It only allows items that meet the specified condition to pass through.

```
Observable<Integer> observable = Observable.range(1, 10);
observable
    .filter(number -> number % 2 == 0)
    .subscribe(System.out::println);
// Output: 2, 4, 6, 8, 10
```

- ***take(n)***: The *take()* operator takes the first *n* items emitted by an Observable and then completes the stream.

```
Observable<Integer> observable = Observable.range(1, 5);
observable
    .take(3)
    .subscribe(System.out::println);
// Output: 1, 2, 3
```

Filtering/suppressing operators

- ***skip(n)***: The *skip()* operator skips the first *n* items emitted by an Observable and emits the rest.

```
Observable<Integer> observable = Observable.range(1, 5);
observable
    .skip(2)
    .subscribe(System.out::println);
// Output: 3, 4, 5
```

- ***takeWhile(predicate)***: The *takeWhile()* operator emits items from an Observable as long as they satisfy a specified condition.

```
Observable<Integer> observable = Observable.range(1, 10);
observable
    .takeWhile(number -> number <= 5)
    .subscribe(System.out::println);
// Output: 1, 2, 3, 4, 5
```

Filtering/suppressing operators

- ***skipWhile(predicate)***: The `skipWhile()` operator skips items emitted by an Observable until a specified condition is no longer met, and then emits the rest.

```
Observable<Integer> observable = Observable.range(1, 10);
observable
    .skipWhile(number -> number <= 5)
    .subscribe(System.out::println);
// Output: 6, 7, 8, 9, 10
```

- ***distinct()***: The `distinct()` operator filters out duplicate items emitted by an Observable, allowing only unique items to be emitted.

```
Observable<Integer> observable = Observable.just(1, 2, 2, 3, 3, 4, 5, 5);
observable
    .distinct()
    .subscribe(System.out::println);
// Output: 1, 2, 3, 4, 5
```

Filtering/suppressing operators

- ***distinctUntilChanged()***: The *distinctUntilChanged()* operator filters out consecutive duplicate items emitted by an *Observable*.

```
Observable<Integer> observable = Observable.just(1, 1, 2, 2, 3, 3, 4, 4, 5)
observable
    .distinctUntilChanged()
    .subscribe(System.out::println);
// Output: 1, 2, 3, 4, 5
```


Transforming operators

- ***map(mapper):*** The `map()` operator transforms each item emitted by an `Observable` using a specified mapping function. It converts items from one type to another or modifies them in some way.

```
Observable<Integer> observable = Observable.just(1, 2, 3);
observable
    .map(number -> number * 2)
    .subscribe(System.out::println);
// Output: 2, 4, 6
```

- ***cast(classType):*** The `cast()` operator attempts to cast the items emitted by an `Observable` to a specific class type. It can be useful when you know that all items in the `Observable` are of a certain type.

```
Observable<Object> observable = Observable.just(1, 2, 3);
observable
    .cast(Integer.class)
    .subscribe(System.out::println);
// Output: 1, 2, 3
```

Transforming operators

- ***sorted()***: The *sorted()* operator arranges the items emitted by an Observable in a sorted order based on their natural order or a custom comparator.

```
Observable<Integer> observable = Observable.just(3, 1, 2, 4);
observable
    .sorted()
    .subscribe(System.out::println);
// Output: 1, 2, 3, 4
```

- ***delay(delayDuration)***: The *delay()* operator introduces a delay before emitting items from an Observable. It can be useful for simulating network latency or scheduling

```
Observable<Integer> observable = Observable.just(1, 2, 3);
observable
    .delay(2, TimeUnit.SECONDS)
    .subscribe(System.out::println);
// Output: (After a 2-second delay) 1, 2, 3
```

Collection operators

- **`toList()`**: The `toList()` operator collects all items emitted by an Observable and assembles them into a single list, which is emitted as a single item when the source Observable completes.

```
Observable<Integer> observable = Observable.just(1, 2, 3, 4, 5);
observable
    .toList()
    .subscribe(list -> System.out.println(list));
// Output: [1, 2, 3, 4, 5]
```

- **`toSortedList()`**: The `toSortedList()` operator collects and sorts the items emitted by an Observable into a sorted list based on their natural order.

```
Observable<Integer> observable = Observable.just(3, 1, 4, 2, 5);
observable
    .toSortedList()
    .subscribe(sortedList -> System.out.println(sortedList));
// Output: [1, 2, 3, 4, 5]
```

Error recovery operators

- ***onErrorReturnItem(defaultValue)***: The `onErrorReturnItem()` operator catches an error emitted by the source Observable and replaces it with a default value before emitting it to the subscriber.

```
Observable<Integer> observable = Observable.create(emitter -> {
    emitter.onNext(1);
    emitter.onError(new RuntimeException("Error"));
});

observable
    .onErrorReturnItem(-1)
    .subscribe(
        item -> System.out.println("Received: " + item),
        error -> System.err.println("Error: " + error)
    );
// Output: Received: -1
```

Error recovery operators

- ***onErrorResumeNext(backupObservable):*** The `onErrorResumeNext()` operator allows you to switch to a backup Observable in case an error occurs in the source Observable. It continues emitting items from the backup Observable.

```
Observable<Integer> original = Observable.create(emitter -> {
    emitter.onNext(1);
    emitter.onError(new RuntimeException("Error"));
});

Observable<Integer> backup = Observable.just(2, 3, 4);

original
    .onErrorResumeNext(backup)
    .subscribe(
        item -> System.out.println("Received: " + item),
        error -> System.err.println("Error: " + error)
    );

// Output: Received: 1, Received: 2, Received: 3, Received: 4
```

Error recovery operators

- ***retry(maxRetries)***: The `retry()` operator allows you to retry the execution of the source Observable a specified number of times when an error occurs. It resubscribes to the source Observable each time it encounters an error.

```
int maxRetries = 2;
Observable<Integer> observable = Observable.create(emitter -> {
    emitter.onNext(1);
    emitter.onError(new RuntimeException("Error"));
});

observable
    .retry(maxRetries)
    .subscribe(
        item -> System.out.println("Received: " + item),
        error -> System.err.println("Error: " + error)
    );

// Output: Received: 1, Received: 1, Received: 1, Error: java.lang.RuntimeException
```

Action operators

- ***doOnSubscribe():*** *This operator is called when an Observer subscribes to an Observable. It's often used to perform actions related to starting the data emission process.*

```
Observable<Integer> observable = Observable.just(1, 2, 3, 4, 5);

observable
    .doOnSubscribe(disposable -> System.out.println("Subscribed"))
    .subscribe(System.out::println);
```

- ***doOnDispose():*** *This operator is called when an Observer unsubscribes or disposes of its subscription. It's used to perform cleanup or stop unnecessary operations.*

```
Disposable disposable = observable
    .doOnDispose(() -> System.out.println("Disposed"))
    .subscribe();

// Later, you can dispose of the subscription
disposable.dispose(); // This will print "Disposed"
```

Action operators

- ***doOnNext():*** *This operator is called every time an item is emitted by the Observable, allowing you to perform actions or logging at each emission point.*

```
Observable<Integer> observable = Observable.just(1, 2, 3, 4, 5);

observable
    .doOnNext(item -> System.out.println("Item emitted: " + item))
    .subscribe();
```

- ***doOnComplete():*** *This operator is called when the Observable completes successfully, indicating that there are no more items to emit.*

```
observable
    .doOnComplete(() -> System.out.println("Completed"))
    .subscribe();
```


Action operators

- **`doOnError()`**: *This operator is called when an error occurs during the processing of the Observable.*

```
Observable<Integer> errorObservable = Observable.error(new RuntimeException("Error"))  
  
errorObservable  
    .doOnError(error -> System.out.println("Error: " + error.getMessage()))  
    .subscribe();
```

- **`doOnSuccess()`**: *This operator is used with Single and Maybe Observables and is called when the Observable completes successfully with a single value.*

```
Single<Integer> single = Single.just(42);  
  
single  
    .doOnSuccess(item -> System.out.println("Success: " + item))  
    .subscribe();
```

Combine multiple types of operators

- This demonstrates the power of RxJava in easily and flexibly combining and processing data.

```
Observable<Integer> observable = Observable.just(1, 2, 3, 4, 5);

observable
    // Filter out even numbers
    .filter(number -> number % 2 == 0)

    // Multiply each number by 10
    .map(number -> number * 10)

    // Sum all the numbers
    .reduce((accumulator, number) -> accumulator + number)

    // Log the result
    .doOnSuccess(result -> System.out.println("Final result: " + result))

    .subscribe();
```

Section 2

Multi Threading using reactive approach



Why Reactive Programming?

The main feature of RxJava when handling multithreading using the reactive approach is the effective management and sequencing of asynchronous tasks. Here are some key benefits of using RxJava for multithreading in Android:

- Easy Multithreading Management
- Avoiding Callback Hell
- Error Handling Made Easy
- Task Cancellation



Why Reactive Programming?

- **Easy Multithreading Management:** RxJava helps you manage and organize multithreaded tasks easily and flexibly.
- **Avoiding Callback Hell:** Using RxJava, you avoid callback hell and replace it with readable and manageable syntax.
- **Error Handling Made Easy:** RxJava provides an easy and intuitive way to handle errors, helping you manage situations when errors occur.
- **Task Cancellation:** RxJava allows for easy cancellation of running tasks, helping manage resources and improving application efficiency.

RxJava is NOT Multi-Threaded by Default

RxJava, by default, is not multi-threaded in any way. The definition given for RxJava on their official website is as follows:

A library for composing asynchronous and event-based programs using observable sequences for the Java VM.

However, it's important to note that RxJava provides the flexibility for you to execute tasks on different threads as needed. This is achieved through operators like **subscribeOn()** and **observeOn()**

Understanding subscribeOn() & observeOn()

- Are two essential operators in RxJava used to manage the execution and processing of data on different threads. Here's a detailed explanation of how they work:
 - **subscribeOn():** is used to specify the thread where Observable tasks will be performed.
 - It affects data emission from the Observable.
 - Typically placed at the beginning of the operator chain to apply to the entire sequence of operations.
 - This helps avoid blocking the main thread of the application and performs heavy tasks on background threads.
 - **observeOn():** is used to specify the thread where the results of tasks will be processed.
 - It affects the processing of data after it has been emitted from the Observable.
 - It can be set multiple times in the operator chain to switch processing threads as needed.
 - It's often used to ensure that task results are processed on the appropriate thread, such as the main thread for updating the user interface.

Befriending The Schedulers

- The main threading in RxJava starts with the powerful set of Schedulers it provides. In RxJava, you cannot directly access or manipulate threads. If you want to play with threads, you have to do it through the in-built Schedulers.
- You can think of Schedulers as **threads or thread pools (a collection of threads) to perform different kinds of jobs.**
- Here are some common Schedulers in RXJava:
 - `Schedulers.io()`
 - `Schedulers.computation()`
 - `Schedulers.newThread()`
 - `Schedulers.from(Executor executor)`
 - `AndroidSchedulers.mainThread()`

Befriending The Schedulers

- **Schedulers.io():** Used for executing I/O tasks such as reading/writing to files or querying databases. This Scheduler utilizes a pool of background threads for handling these tasks.
- **Schedulers.computation():** Used for lightweight computational tasks, suitable for arithmetic operations and numeric data processing. It uses a computation thread pool.
- **Schedulers.newThread():** Creates a new thread for each task. It's not a pool, so it's not suitable for performance-intensive tasks.
- **Schedulers.from(Executor executor):** This method enables you to build a custom Scheduler using your own Executor, which manages task execution on various threads
- **AndroidSchedulers.mainThread():** Ensures that tasks are executed on the main thread of the Android application, typically used for updating the user interface.

Section 2

DEMO

Section 3

Kotlin Flow replacement

Why Consider Alternatives?

- **Challenges with RxJava:** RxJava has been a popular choice for handling asynchronous programming in Android, but it comes with its set of complexities and a steep learning curve.
- **Evolving Kotlin Ecosystem:** The Kotlin programming language has gained immense popularity in the Android community, and Kotlin Flow is a natural fit for Kotlin-based projects.
- **Simplicity and Conciseness:** Kotlin Flow offers a more concise and intuitive syntax compared to RxJava, making it easier for developers to work with.
- **Performance and Efficiency:** Kotlin Flow is designed with modern Android development in mind, focusing on performance improvements and reduced overhead.
- **Compatibility:** Kotlin Flow integrates seamlessly with existing Kotlin codebases and Android's coroutine support.

Meet Kotlin Flow

- **What is Kotlin Flow:** Kotlin Flow is a modern asynchronous programming library for Kotlin that simplifies working with asynchronous data streams. It's part of Kotlin's standard library and is designed to replace RxJava for Kotlin-based Android projects.
- **Key Features:** Kotlin Flow offers several key features, including:
 - *Coroutines Integration:* It's built on top of Kotlin's coroutines, which are officially supported by Android. This simplifies concurrency management.
 - *Sequential and Concise:* Kotlin Flow provides a sequential and straightforward API for handling asynchronous operations.
 - *Cancellation Support:* It fully supports the cancellation of operations, helping to prevent resource leaks.
 - *Exception Handling:* Kotlin Flow makes it easy to handle exceptions within the flow.
 - *Back Pressure:* It offers built-in support for back pressure, allowing control over data flow in reactive streams.
- **Why Kotlin Flow:** Kotlin Flow's seamless integration with Kotlin and coroutines, along with its simplicity and efficiency, make it an attractive alternative to RxJava for Android development.

Key Differences Between Flow and RxJava

- **Syntax and API:** One of the primary differences is in syntax and API design. Kotlin Flow leverages Kotlin's language features, resulting in a more concise and Kotlin-native syntax. RxJava, on the other hand, has its own set of operators and syntax.
- **Cancellation and Exception Handling:** Kotlin Flow has first-class support for cancellation and exception handling through Kotlin's coroutines. This simplifies resource management and error handling compared to RxJava.
- **Coroutines Integration:** Kotlin Flow seamlessly integrates with Kotlin's coroutines, which are officially supported in Android. RxJava, while compatible with coroutines, is a separate library.
- **Back Pressure:** RxJava provides explicit support for back pressure, which can be useful in certain scenarios where data flow control is critical. Kotlin Flow offers a simpler approach to back pressure.
- **Performance:** Kotlin Flow aims to be more efficient in terms of memory usage and performance, particularly in Android app scenarios.

Getting Started with Kotlin Flow

- **Flow:** Flow is an asynchronous data stream in Kotlin Flow. You can create a Flow to emit data over time. To create a Flow, you use the `flow { ... }` function and use the `emit` operator to emit values.

```
fun simpleFlow(): Flow<Int> = flow {  
    for (i in 1..5) {  
        delay(1000) // Simulate some asynchronous work  
        emit(i)      // Emit values to the flow  
    }  
}
```

- **Operators:** Kotlin Flow provides many operators to transform, filter, and combine data streams. These operators allow you to perform tasks such as mapping, filtering, zipping, aggregating, and more

Getting Started with Kotlin Flow

- **Extension Functions:** Kotlin Flow comes with numerous extension functions that allow you to add functionality to a Flow.

```
fun getUserFlow(): Flow<User> = flow {  
    val userData = ApiService.getUserData()  
    emit(userData)  
}.flowOn(Dispatchers.IO)
```

- **Coroutine Scope:** Kotlin Flow is typically used within a Coroutine Scope to manage the execution context. You can use `runBlocking`, `lifecycleScope`, or `viewModelScope` depending on your application's environment.

```
fun main() = runBlocking {  
    simpleFlow()  
        .collect { value ->  
            println(value) // Process emitted values  
        }  
}
```


Getting Started with Kotlin Flow

- **Collect Function:** The collect function is a primary method for processing values from a Flow. You use it to register a listener and process values as they are emitted from the Flow.

```
simpleFlow()
    .collect { value ->
        println(value) // Process emitted values
    }
```

- **Suspending Functions:** Inside your functions or methods, you can use suspending functions to await values from a Flow or perform other asynchronous tasks without blocking the main thread.

```
suspend fun fetchAndProcessData() {
    val data = simpleFlow().toList()
    // Process the collected data
}
```

Replacing RxJava with Kotlin Flow

- RxJava Code (Before):

```
Observable<User> getUserObservable() {  
    return ApiService.getUserData()  
        .subscribeOn(Schedulers.io())  
        .observeOn(AndroidSchedulers.mainThread());  
}  
  
void fetchUser() {  
    getUserObservable()  
        .subscribe(user -> {  
            // Process user data and update UI  
            updateUI(user);  
        }, error -> {  
            // Handle errors  
            showError(error);  
        });  
}
```

Replacing RxJava with Kotlin Flow

- Kotlin Flow Code (After):

```
suspend fun getUserFlow(): Flow<User> = flow {  
    val userData = ApiService.getUserData()  
    emit(userData)  
}.flowOn(Dispatchers.IO)  
  
suspend fun fetchUser() {  
    getUserFlow()  
        .catch { error ->  
            // Handle errors  
            showError(error)  
        }  
        .collect { user ->  
            // Process user data and update UI  
            updateUI(user)  
        }  
}
```

THANK YOU!

